STAT 4710/5710: Homework 1

Ying Jin

Due: September 16, 2025 at 11:59am

Contents

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

> "Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others' solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty."

In accordance with this policy,

Please disclose all classmates with whom you collaborated:

Please disclose which AI tools you used, and how you used them:

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use 'homework-1-template.Rmd' as a starting point for your writeup, adding your solutions after "**Solution**". Add your R code using code chunks and add your text answers using **bold text**. Consult the preparing reports guide for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to "print a tibble", you should print the tibble directly.

Programming

Apart from the "R basics" part, the `tidyverse` paradigm for data visualization, manipulation, and wrangling is **required**. Codes written in base R may receive deductions at the discretion of the instructor, since the purpose of assignments is to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you must:

- Use `pandas` (or similar) for data wrangling with method chaining (`groupby`, `merge`, `assign`, etc.) rather than raw loops.
- Use `plotnine` for visualization (preferred, since it parallels ggplot2). If using seaborn/matplotlib, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).
- Present both code chunks and the corresponding outputs/interpretations in your PDF.

The deduction rules will be similar to those for R.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels**: Always include informative axis labels and, if appropriate, titles.
- **Required elements**: If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity**: Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

1 R basics (10 points)

1.1 Creating vectors of divisible integers (5 points)

Use the "`seq`" function to create a vector called `vec1` consisting of all integers divisible by 13 between 0 and 200 (including 0), in DESCENDING order. Put in the code chunk and print out the vector `vec1`. Hint: revert a vector using `rev`.

1.2 Creating repeated vectors (5 points)

- Try the following code to learn what the `rep` function does (Write a chunk, copy-paste the code, and print the results):

```
rep(c(1,2), times=4)
rep(c(0,1), times=c(3,5)
```

- Create a vector called `vec2` consisting of all odd numbers between 101 and 149 in ascending order, with the K-th number repeated K times: $(101, 103, 103, 105, 105, 105, \ldots)$. Put in the code chunk and print out the vector `vec2`. Hint: Use `seq` and `rep` and `:` within a single line.

Case study: NYC Flights 2013

In this case study, we will analyze a random subset of all flights departing NYC (JFK, LGA, EWR) in 2013. The dataset contains information about:

- `year`, `month`, `day` — date of departure
- `dep_delay`, `arr_delay` — departure and arrival delays (minutes)
- `air_time`, `distance` — flight duration and distance
- `carrier` — two-letter airline code
- `origin`, `dest` — airports for departure and destinations

We'll investigate patterns in flight delays and efficiency using data wrangling, summarizing, and visualization.

2 Processing (45 points)

2.1 Import (5 points)

- (3 points) Import the data `NYCflights13sub.csv` into a `tibble` called `flight_raw` and print it out. How many rows and columns does the data have?
- (2 points) Which variables are numeric, and which are categorical? Name 3 variables for each.

2.2 Tidy: missing values (5 points)

Some values in the raw data are missing, which we will discard.

- (3 points) Create a new tibble `flights_clean` by removing rows with missing values in `dep_delay`, `arr_delay`, or `air_time`. Print the new tibble.
- (2 points) How many rows were dropped due to missingness?

2.3 Tidy: filtering major carriers (15 points)

In the cleaned data, we will only focus on carriers with sufficiently many data. This requires us to find out these carriers and preserve only data from these carriers.

- (5 points) Create a new tibble `carrier_counts` that counts the number of flights for each carrier in `flights_clean`, and print it out.

- (5 points) Create a vector `main_carriers` that contains the carrier codes of all carriers with at least 1000 flights in `flights_clean`, and print it out.
- (5 points) Create a new tibble `flights_clean` by filtering `flights_clean` to only include flights from the carriers in `main_carriers`, and print it out. How many rows were further dropped due to this filtering?

From now on, we will work with the tibble `flights_clean`.

2.4 Quality control (20 points)

It's always a good idea to check whether a dataset is internally consistent. In this case, we are given both the scheduled time and the actual time, as well as the delayed time, for departure, so we can check whether these match. To this end, carry out the following steps:

- (5 points) Look at the columns `dep_time` which is the actual departure time of the flight, `sched_dep_time` which is the scheduled departure time of the flight, and `dep_delay` which is the departure delay in minutes. How should we interpret the 3 or 4 digits in `dep_time` and `sched_dep_time`?
- (5 points) Create new columns `dep_time_hr` and `dep_time_min` in `flights_clean` that converts `dep_time` into the hour and minute of the given time (e.g., 1130 to 11 and 30). Similarly, create new columns `sched_dep_time_hr` and `sched_dep_time_min` in `flights_clean` that converts `sched_dep_time` into the hour and minute of the given time.
- (5 points) Using the constructed columns `dep_time_hr`, `dep_time_min`, `sched_dep_time_hr`, and `sched_dep_time_min`, create a new column `dep_delay_computed` that computes the departure delay in minutes.
- (5 points) Create a scatter plot of `dep_delay_computed` versus `dep_delay`, including a 45° line. Comment on the relationship between the computed and provided delay time statistics. Did you find any discrepancies? If yes, print out the `dep_time`, `sched_dep_time`, `dep_delay`, `dep_delay_computed` columns of an observation with discrepancy, and explain why it occurs.

3 Explore (45 points)

Now that the data are in tidy format, we can explore them by producing visualizations and summary statistics. We will use the columns originally provided in the dataset.

3.1 Departure and arrival delays (15 points)

Are there any relationship between departure and arrival delay times? This can be studied via visualization.

- (5 points) Create a scatter plot of `arr_delay` versus `dep_delay`, adding a 45° line.
- (5 points) In the same plot, add a smooth curve to the scatter plot using `geom_smooth()`. Comment on the relationship between departure and arrival delay times.
- (5 points) Create a scatter plot of `arr_delay` versus `dep_delay`, faceting by the origin airport `origin`. Comment on the patterns.

3.2 Delays by month and by carrier (20 points)

We now explore the delay pattern by month and carrier. We will only focus on some main carriers with sufficiently many data.

- (5 points) Create a new tibble called `delay_aggregate` by computing the mean arrival delay time for each month and each carrier, stored in the column named `delay_month`. Print out the new tibble.
- (5 points) Create a new tibble called `delay_yr` by computing the overall mean arrival delay time for each carrier, stored in the column named `delay_by_year`. Print out the new tibble.

- (5 points) Join `delay_aggregate` and `delay_yr` into the tibble `delay_aggregate`, so that each row contains the per-month mean arrival delay time as well as the overall mean arrival delay time for that carrier. Print out the new tibble.
- (5 points) For each carrier, plot monthly mean arrival delay `(y = delay_month, x = month)` as a line with points, and facet by carrier (one panel per carrier). Add a red dashed horizontal line at that carrier's overall mean arrival delay.

3.3 Carrier efficiency (5 points)

We define a simple efficiency score for each carrier:

$$\text{efficiency} = \frac{\text{on-time rate}}{1 + \text{mean departure delay (positive-only)}}.$$

where on-time rate is the proportion of flights with non-positive arrival delay, and mean departure delay (positive) is the average `dep_delay` of the carrier in minutes, *after we clip negatives at 0 so early departures don't inflate the score.*

- (5 points) Based on `flights_clean`, create a new tibble called `efficiency_carrier` and compute the efficiency per carrier, on-time rate, and mean departure delay (column names `efficiency`, `on_time_rate`, and `mean_dep_delay`). Print out the new tibble.
- (5 points) Use `tidyverse` to find and print out the top 3 carriers with highest efficiency. Which carriers are they?